

How To Use AWK to Manipulate Text in Linux

Updated on May 12, 2026

[Linux Commands](#)

[Linux Basics](#)

[System Tools](#)



By [Justin Ellingwood](#), [Brian Hogan](#) and [Manikandan Kurup](#)



[Table of contents](#)



[Popular topics](#)



Introduction



[Linux](#) utilities often follow the [Unix](#) philosophy of design. Tools are encouraged to be small, use plain text files for input and output, and operate in a modular manner. Because of this legacy, Linux provides powerful text processing tools like [sed](#) and [awk](#) .

`awk` is both a programming language and a text processor, which makes it especially useful for working with structured, line-oriented data like logs, tables, and simple delimited files. In this guide, you will start with the basic pattern-and-action syntax, learn how it compares to `sed` for common text-processing tasks, and then build up to field-based filtering, internal variables, and formatted output.

From there, you will use associative arrays to count, group, and aggregate values in one pass, combine `awk` with other commands in pipelines, and embed `awk` in shell scripts for repeatable automation. You will also walk through real-world parsing examples and end with FAQs that address common questions about `awk`.

Key Takeaways:

- Use `awk`'s pattern-and-action model to match the lines you care about and transform only those records.
- Treat input as columns by default: use `$1`, `$2`, and friends for fields, and use `$0` when you need the entire record for printing or further matching.
- Set `FS` / `OFS` up front so delimited input (like `/etc/passwd` or CSV-like data) can be parsed and printed consistently.
- Use `BEGIN` for one-time setup (like setting separators or printing headers) and `END` for summary output (like totals or rollups).
- Lean on `NR`, `NF`, `FILENAME`, and `FNR` to write record-aware logic, debug unexpected input, and handle multiple files cleanly.
- Reach for associative arrays when you need fast counting, grouping, summing, or other one-pass aggregation without a separate `sort` / `uniq` step.
- Prefer `-v` when passing shell values into `awk` so quoting, spaces, and special characters do not break your program.
- Choose `awk` for streaming, line-oriented text and quick reports, but switch tools when parsing rules get complex (for example, CSV with quoted commas).

AWK vs sed: When Should You Use Each?

Both `awk` and `sed` are standard Unix text-processing tools that are often mentioned together because they operate on text streams. However, they are designed with different goals in mind, and choosing the right one depends on the type of task you

are trying to accomplish.

At a high level, `sed` is a stream editor that excels at making simple, line-oriented transformations, while `awk` is a pattern-scanning and data-processing language that is better suited for working with structured or column-based data.

Key Differences

Feature	<code>awk</code>	<code>sed</code>
Primary purpose	Pattern scanning and data processing	Stream editing and text transformation
Data handling	Field- and column-based	Line-based
Programming capability	Full scripting language (variables, loops, arrays)	Limited scripting (basic commands)
Best suited for	Structured data (CSV, logs, tables)	Simple substitutions and edits
Readability for complex tasks	Higher	Lower

When to Use `sed`

You should use `sed` when your task involves straightforward transformations applied to lines of text. It is especially useful when you want to modify text quickly without needing to understand its internal structure.

For example, if you want to replace all occurrences of the word "error" with "warning" in a file, `sed` provides a concise and efficient solution:

```
$ sed 's/error/warning/g' file.txt
```

In general, `sed` is a good choice when you need to:

- Perform find-and-replace operations.
- Delete or insert specific lines.
- Apply simple transformations to entire lines of text.

Because `sed` operates on one line at a time and does not inherently understand fields or columns, it is less suitable for tasks that involve structured data.

When to Use `awk`

You should use `awk` when your task involves extracting, analyzing, or transforming structured data. Unlike `sed`, `awk` understands the concept of fields and can easily work with columns in a file.

For example, if you want to print only the first and third columns from a file, `awk` makes this straightforward:

```
$ awk '{print $1, $3}' file.txt
```

`awk` is particularly useful when you need to:

- Extract specific columns from tabular data.
- Filter rows based on conditions.
- Perform calculations or aggregations.
- Generate formatted output or reports.

Because `awk` includes programming constructs such as variables, conditions, and loops, it can handle more complex data-processing tasks than `sed`.

Quick Decision Guide

If you are unsure which tool to use, the following guidelines can help:

- Use `sed` when your task involves simple, line-based text transformations and does not require awareness of fields or structure.
- Use `awk` when your task involves structured data, requires working with columns, or needs conditional logic and data processing.

Example: Comparing Approaches

Consider a file where each line contains multiple space-separated fields, and you want to extract only the first column.

With `awk`, this task is simple and intuitive:

```
$ awk '{print $1}' file.txt
```

With `sed`, achieving the same result requires a more complex regular expression:

```
$ sed 's/^\([^ ]*\).*$/\1/' file.txt
```

While both commands produce the same output, the `awk` version is easier to read and better reflects the structure of the data. This highlights a key distinction: `awk` is designed for working with fields, whereas `sed` operates purely on text patterns.

Although `sed` and `awk` are both powerful tools for processing text, they are optimized for different use cases. `sed` is ideal for quick and simple text transformations, while `awk` is better suited for structured data processing and more complex logic. Understanding this distinction will help you choose the most efficient tool for your specific task.

Basic Syntax

The `awk` command is included by default in all modern Linux systems, so you do not need to install it to begin using it.

`awk` is most useful when handling text files that are formatted in a predictable way. For instance, it is excellent at parsing and manipulating tabular data. It operates on a line-by-line basis and iterates through the entire file.

By default, it uses whitespace (spaces, tabs, etc.) to separate fields. Luckily, many configuration files on your Linux system use this format.

The basic format of an `awk` command is:

```
$ awk '/ search_pattern / { action_to_take_on_matches ; another_action ; }
```

You can omit either the search portion or the action portion from any `awk` command. By default, the action taken if the "action" portion is not given is "print". This simply prints all lines that match.

If the search portion is not given, `awk` performs the action listed on each line.

If both are given, `awk` uses the search portion to decide if the current line reflects the pattern, and then performs the actions on matches.

In its simplest form, you can use `awk` like `cat` to print all lines of a text file out to the screen.

Create a `favorite_food.txt` file that lists the favorite foods of a group of friends:

```
echo "carrot sandy  
wasabi luke  
sandwich brian  
salad ryan  
spaghetti jessica" > favorite_food.txt
```

Now use the `awk` command to print the file to the screen:

```
$ awk '{print}' favorite_food.txt
```

You'll see the file printed to the screen:

Output

```
carrot sandy
wasabi luke
sandwich brian
salad ryan
spaghetti jessica
```

This isn't very useful. Let's try out `awk`'s search filtering capabilities by searching through the file for the text "sand":

```
$ awk '/sand/' favorite_food.txt
```

Output

```
carrot sandy
sandwich brian
```

As you can see, `awk` now only prints the lines that have the characters "sand" in them.

Using regular expressions, you can target specific parts of the text. To display only the line that starts with the letters "sand", use the regular expression `^sand` :

```
$ awk '/^sand/' favorite_food.txt
```

This time, only one line is displayed:

```
Output  
sandwich brian
```

Similarly, you can use the action section to specify which pieces of information you want to print. For instance, to print only the first column, use the following command:

```
$ awk '/^sand/ {print $1;}' favorite_food.txt
```

```
Output  
sandwich
```

You can reference every column (as delimited by whitespace) by variables associated with their column number. For example, the first column is `$1` , the second is `$2` , and you can reference the entire line with `$0` .

Internal Variables and Expanded Format

The `awk` command uses some internal variables to assign certain pieces of information as it processes a file.

The internal variables that `awk` uses are:

- **FILENAME:** References the current input file.
- **FNR:** References the number of the current record relative to the current input file. For instance, if you have two input files, this would tell you the record number in each file rather than the total record number.

- **FS:** The current field separator used to denote each field in a record. By default, this is set to whitespace.
- **NF:** The number of fields in the current record.
- **NR:** The number of the current record.
- **OFS:** The field separator for the output data. By default, this is set to whitespace.
- **ORS:** The record separator for the output data. By default, this is a newline character.
- **RS:** The record separator used to distinguish separate records in the input file. By default, this is a newline character.

You can change the values of these variables at will to match the needs of your files. Usually you do this during the initialization phase of your processing.

For example, to print each line along with its line number, you can use:

```
$ awk '{print NR, $0}' file.txt
```

To display how many fields each line contains:

```
$ awk '{print "Fields:", NF}' file.txt
```

This brings us to another important concept. The `awk` syntax is slightly more complex than what you've used so far. There are also optional `BEGIN` and `END` blocks that can contain commands to execute before and after the file processing, respectively.

This makes our expanded syntax look something like this:

```
awk 'BEGIN { action; }  
/search/ { action; }
```

```
END { action; }' input_file
```

The `BEGIN` and `END` keywords are specific sets of conditions, just like the search parameters. They match before and after the document has been processed.

This means that you can change some of the internal variables in the `BEGIN` section. For instance, the `/etc/passwd` file is delimited with colons (`:`) instead of whitespace.

To print out the first column of this file, execute the following command:

```
awk 'BEGIN { FS=":"; }  
{ print $1; }' /etc/passwd
```

Output

```
root  
daemon  
bin  
sys  
sync  
games  
man  
. . .
```

You can use the `BEGIN` and `END` blocks to print information about the fields you are printing. Use the following command to transform the data from the file into a table, nicely spaced with tabs using `\t` :

```
awk 'BEGIN { FS=":"; print "User\t\tUID\t\tGID\t\tHome\t\tShell\n-----"  
{print $1,"\t\t",$3,"\t\t",$4,"\t\t",$6,"\t\t",$7;}  
END { print "-----\nFile Complete" }' /etc/passwd
```

You'll see this output:

Output

User	UID	GID	Home	Shell
root	0	0	/root	/bin/bash
daemon	1	1	/usr/sbin	/bin/sh
bin	2	2	/bin	/bin/sh
sys	3	3	/dev	/bin/sh
sync	4	65534	/bin	/bin/sync
. . .				

File Complete

As you can see, you can format things quite nicely by taking advantage of some of `awk`'s features.

Each of the expanded sections is optional. In fact, the main action section itself is optional if another section is defined. For example, you can do things like this:

```
$ awk 'BEGIN { print "We can use awk like the echo command"; }'
```

And you'll see this output:

Output

```
We can use awk like the echo command
```

Now let's look at how to search for text within fields of the output.

Field Searching and Compound Expressions

In one of the previous examples, you printed the line in the `favorite_food.txt` file that began with "sand". This was easy because you were looking for the beginning of the entire line.

What if you wanted to find out if a search pattern matched at the beginning of a *field* instead?

Create a new version of the `favorite_food.txt` file that adds an item number in front of each person's food:

```
echo "1 carrot sandy
2 wasabi luke
3 sandwich brian
4 salad ryan
5 spaghetti jessica" > favorite_food.txt
```

If you want to find all foods from this file that begin with "sa", you might begin by trying something like this:

```
$ awk '/sa/' favorite_food.txt
```

This shows all lines that contain "sa":

```
Output
1 carrot sandy
2 wasabi luke
3 sandwich brian
4 salad ryan
```

Here, you are matching any instance of "sa" in the word. This ends up including things like "wasabi" which has the pattern in the middle, or "sandy" which is not in the column you want. In this case, you are only interested in words beginning with "sa" in

column you want. In this case, you are only interested in words *beginning* with "sa" in the *second* column.

You can tell `awk` to only match at the beginning of the second column by using this command:

```
$ awk '$2 ~ /^sa/' favorite_food.txt
```

This allows you to search only at the beginning of the second column for a match.

The `field_num ~` part specifies that `awk` should only pay attention to the second column.

Output

```
3 sandwich brian
4 salad ryan
```

You can just as easily search for things that do *not* match by including the `!"` character before the tilde (`~`). This command will return all lines that do *not* have a food that starts with "sa":

```
$ awk '$2 !~ /^sa/' favorite_food.txt
```

Output

```
1 carrot sandy
2 wasabi luke
5 spaghetti jessica
```

If you want to find lines where the food does not start with "sa" and the item number is less than 5, you can use a compound expression like this:

```
$ awk '$2 !~ /^sa/ && $1 < 5' favorite_food.txt
```

This introduces a few new concepts. The first is the ability to add additional requirements for the line to match by using the `&&` operator. Using this, you can combine an arbitrary number of conditions for the line to match. In this case, you are using this operator to add a check that the value of the first column is less than 5.

You'll see this output:

Output

```
1 carrot sandy
2 wasabi luke
```

You can use `awk` to process files, but you can also work with the output of other programs.

Using Associative Arrays for Data Aggregation

So far, you have used `awk` to filter and print data. One of the features that makes `awk` especially useful for real-world text processing is its support for **associative arrays**, which let you store values while you scan through a file.

Unlike traditional arrays that use numeric indices, associative arrays in `awk` use **string keys**. This makes them a good fit for tasks like counting, grouping, and summarizing records, because you can use a field value (like a username or department) as the key.

Counting Occurrences

A common use case for associative arrays is counting how many times a value

... comment and use the `END` block to print the final counts. The `END` block appears in a file. In this pattern, you increment a counter for each key as you read each line, and then print the final counts in an `END` block.

Consider a file called `fruits.txt` :

```
echo "apple  
banana  
apple  
orange  
banana  
apple" > fruits.txt
```

You can count how many times each fruit appears using:

```
$ awk '{count[$1]++} END {for (item in count) print item, count[item]}' fruits.txt
```

This produces output similar to:

```
apple 3  
banana 2  
orange 1
```

In this example:

- `count[$1]++` increments a counter for each unique value in the first column.
- `$1` is used as the **key** in the associative array.
- The `END` block prints the aggregated results after the file has been read.

Depending on your `awk` implementation, the `for (item in count)` loop may print keys in an arbitrary order. If you need sorted output, you can pipe the results to `sort`.

Grouping Data by Field

Associative arrays can also be used to group related data. Instead of storing a numeric count, you can build up a string (or other value) per key as you process each record.

Suppose you have a file `employees.txt` :

```
echo "HR Alice
IT Bob
HR Charlie
IT David
Finance Emma" > employees.txt
```

You can group employees by department:

```
$ awk '{group[$1] = (group[$1] ? group[$1] " " : "") $2} END {for (dept in group) print dept, group[dept]}
```

Example output:

```
HR: Alice Charlie
IT: Bob David
Finance: Emma
```

Here:

- `$1` (department) is used as the key.
- `$2` (employee name) is appended to the corresponding group value.

This approach avoids a leading space for the first employee in each department, which makes the output easier to read

which makes the output easier to read.

Summing Values

You can also use associative arrays to calculate totals. This is a common pattern when your file contains a label in one field and a numeric value in another field.

For example, consider a file `sales.txt` :

```
echo "Alice 100  
Bob 200  
Alice 150  
Bob 50" > sales.txt
```

To calculate total sales per person:

```
$ awk '{sum[$1] += $2} END {for (name in sum) print name, sum[name]}' sales.txt
```

Output:

```
Alice 250  
Bob 250
```

In this case:

- `$1` is the key (name).
- `$2` is added to the running total for that key.

Finding the Maximum Value

Associative arrays can also help you identify maximum or minimum values. A typical workflow is to build an associative array first (such as totals per person), and then

workflow is to build an aggregate array first (such as totals per person), and then scan that array to find the largest value.

```
awk '{sum[$1] += $2} END {  
    for (name in sum) {  
        if (top == "" || sum[name] > max) { max = sum[name]; top = name }  
    }  
    print "Top performer:", top, max  
}' sales.txt
```

This allows you to compute insights directly within `awk`, without needing external tools.

Why Associative Arrays Matter

Associative arrays transform `awk` from a simple text-filtering tool into a **lightweight data-processing engine**. With associative arrays, you can accumulate state as you read a file and produce summary output at the end.

With them, you can:

- Build frequency distributions.
- Group and reorganize data.
- Compute aggregates like sums and counts.
- Perform basic analytics directly at the command line.

These capabilities are especially useful when working with logs, CSV files, or system-generated data.

Processing Output from Other Programs

You can use the `awk` command to parse the output of other programs rather than specifying a filename. For example, you can use `awk` to parse out the [IPv4](#) address from the `ip` command.

The `ip a` command displays the IP address, broadcast address, and other information about all the network interfaces on your machine. To display the

information for the interface called `eth0`, use this command:

```
$ ip a s eth0
```

You'll see the following results:

Output

```
2571: eth0@if2572: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue  
    link/ether 02:42:ac:11:00:0b brd ff:ff:ff:ff:ff:ff link-netnsid 0  
    inet 172.17.0.11/16 brd 172.17.255.255 scope global eth0  
        valid_lft forever preferred_lft forever
```

You can use `awk` to target the `inet` line and then print out just the IP address:

```
$ ip a s eth0 | awk -F '[\\ / ]+' '/inet / {print $3}'
```

The `-F` flag tells `awk` to delimit by forward slashes or spaces using the regular expression `[\\ / ]+`. This splits the line `inet 172.17.0.11/16` into separate fields. The IP address is in the third field because the spaces at the start of the line also count as a field, since you delimited by spaces as well as slashes. Note that `awk` treated consecutive spaces as a single space in this case.

The output shows the IP address:

Output

```
172.17.0.11
```

You'll find many places where you can use `awk` to search or parse the output of other commands.

Using AWK in Shell Scripts and Automation

So far, you have used `awk` as a standalone command. In practice, `awk` becomes even more useful when you embed it in shell scripts to automate repetitive data-processing tasks.

By combining `awk` with Bash, you can build lightweight automation workflows that parse logs, extract metrics, and generate reports without adding extra dependencies.

Embedding AWK in a Bash Script

You can include `awk` commands directly inside a shell script just like any other command. This approach is useful when you want to reuse the same extraction logic in multiple places or run it on a schedule.

For example, create a script called `extract_users.sh` :

```
#!/bin/bash

awk -F ":" '{print $1}' /etc/passwd
```

Make the script executable and run it:

```
$ chmod +x extract_users.sh
$ ./extract_users.sh
```

This script extracts and prints all usernames from the `/etc/passwd` file. While simple, it demonstrates how `awk` can be integrated into reusable scripts.

It also shows a common pattern you will use often: `-F ":"` sets the field separator,

and `print $1` prints the first field from each line.

Passing Variables from Bash to AWK

You can pass variables from your shell script into `awk` using the `-v` option. This is useful when you want to make your scripts dynamic.

```
#!/bin/bash

threshold=100

awk -v limit="$threshold" '$2 > limit {print $1, $2}' sales.txt
```

In this example:

- The Bash variable `threshold` is passed into `awk` as `limit`.
- `awk` filters and prints only rows where the second column exceeds the threshold.

This approach allows you to control `awk` behavior from your script logic.

As a best practice, it is generally safer to pass shell variables into `awk` with `-v` than to try to concatenate them directly into the `awk` program.

Processing Files in Bulk

Shell scripts are often used to process multiple files. You can combine loops with `awk` to handle batches of data.

```
#!/bin/bash

for file in *.log; do
    echo "Processing $file"
    awk '/ERROR/ {print $0}' "$file"
done
```

This script iterates over all `.log` files in the directory, uses `awk` to extract lines containing "ERROR", and prints the matching lines for each file.

If you are using Bash and want the loop to skip cleanly when no files match `*.log`, you can enable `nullglob` (for example, `shopt -s nullglob`) so that the pattern expands to nothing instead of the literal string `*.log`.

Automating Log Analysis

A common real-world use case is analyzing logs to extract useful information.

Suppose you want to count how many errors occurred in a log file:

```
#!/bin/bash

logfile="app.log"

awk '/ERROR/ {count++} END {print "Total errors:", count+0}' "$logfile"
```

This script scans the log file for lines containing "ERROR", maintains a running count as it reads the file, and prints the total in an `END` block.

The `count+0` ensures that the script prints `0` even if no lines match.

You can extend this further to:

- Trigger alerts.
- Generate daily summaries.
- Feed results into monitoring systems.

Combining AWK with Other Commands

`awk` is often used in pipelines with other Unix tools to build flexible workflows.

```
$ grep "404" access.log | awk '{print $1}' | sort | uniq -c
```

This pipeline filters HTTP 404 errors, extracts the IP address from the first field, and counts how many times each address appears.

Although `awk` can often replace parts of this pipeline, combining tools can sometimes make scripts easier to understand and maintain.

Why Use AWK in Automation?

Using `awk` inside shell scripts allows you to:

- Eliminate manual data processing.
- Build repeatable workflows.
- Quickly extract insights from system data.
- Avoid heavier scripting languages for simple tasks.

Because `awk` is available on almost all Unix-like systems, scripts that rely on it are also highly portable.

Real-World Examples: Log Parsing and Data Extraction

While the previous examples demonstrate how `awk` works, its real strength lies in solving practical problems. System administrators and developers frequently use `awk` to parse logs, extract structured data, and generate quick reports directly from the command line.

The following examples illustrate how `awk` can be applied to common real-world scenarios.

Parsing Apache Access Logs

Web server logs are one of the most common sources of structured text data. In many default Apache configurations, access logs are written in the “combined” format, which places the client IP address at the start of the line and the HTTP status code later in the record.

Consider a typical Apache access log entry:

```
127.0.0.1 - - [10/Oct/2023:13:55:36 +0000] "GET /index.html HTTP/1.1" 200 1234
```

If you want to extract the IP address and HTTP status code from each request, you can use:

```
$ awk '{print $1, $9}' access.log
```

This works because `$1` represents the client IP address and `$9` represents the HTTP status code in the default access log layout shown above.

To count how many requests resulted in a `404` error:

```
$ awk '$9 == 404 {count++} END {print "404 errors:", count+0}' access.log
```

The `count+0` ensures that the output is `0` even when no lines match.

Extracting Data from CSV Files

Although `awk` defaults to whitespace as a field separator, it can easily handle CSV files by changing the delimiter.

Consider a file `data.csv` :

```
name,age,city
Alice,30,New York
Bob,25,London
Charlie,35,Sydney
```


To print only the names and cities:

```
$ awk -F ',' '{print $1, $3}' data.csv
```

This example assumes that the CSV file does not contain quoted fields with embedded commas. If your data includes quoted commas, you will need a CSV-aware parser.

To filter records where age is greater than 30:

```
$ awk -F ',' '$2 > 30 {print $1, $2}' data.csv
```

This demonstrates how `awk` can be used to query structured datasets without needing a database.

Analyzing System Files

Many Linux system files follow predictable formats, making them ideal for processing with `awk`.

For example, to list all users with a UID greater than 1000 from `/etc/passwd`:

```
$ awk -F ':' '$3 > 1000 {print $1, $3}' /etc/passwd
```

To count how many such users exist:

```
$ awk -F ':' '$3 > 1000 {count++} END {print count+0}' /etc/passwd
```

This prints `0` if no user accounts match the condition.

Summarizing Log Data

You can combine multiple `awk` features to generate quick summaries.

For example, to count how many requests were made by each IP address:

```
$ awk '{count[$1]++} END {for (ip in count) print ip, count[ip]}' access.log
```

This produces a frequency distribution of requests per client. Keep in mind that `for (ip in count)` may print results in an arbitrary order, so you can pipe the output to `sort` if you need consistent ordering.

By applying `awk` in these contexts, you can automate tasks that would otherwise require more complex tools or manual effort.

FAQs

1. What is the difference between AWK, gawk, and mawk?

`awk` is the POSIX-defined language and command interface. `gawk` (GNU Awk) is a common default on many Linux distributions and includes extra features beyond POSIX. `mawk` is another implementation that often prioritizes speed and low overhead, but it may not support the same GNU-specific extensions as `gawk`.

2. How do I specify a custom field separator in AWK?

You can set a custom field separator with the `-F` option, which tells `awk` how to split each input line into fields. For example, `awk -F ':' '{print $1}' /etc/passwd` uses a colon delimiter and prints the first field (the username) from each line.

3. What is the difference between AWK and sed?

`sed` is a stream editor that is best for line-oriented transformations such as substitutions, deletions, and inserts. `awk` is a pattern-scanning language that is designed for field-based processing, conditional logic, and simple calculations. In general, `awk` is a better fit for column-based data and reporting, while `sed` is often more direct for straightforward text substitutions.

4. How do I use AWK to count the number of occurrences of a word or value?

You can use an associative array as a counter, incrementing the value for each key as you read the file, then printing the results in an `END` block. For example, `awk '{count[$1]++} END {for (w in count) print w, count[w]}' file.txt` counts unique values in the first field.

5. Can AWK handle multi-line records?

Yes. If you set the record separator `RS` to an empty string (`RS=""`), `awk` treats blank-line-separated blocks as a single record. This is useful when you are processing input that is structured in paragraphs or blocks rather than one record per line.

6. How do I pass a shell variable into an AWK command?

You can pass shell variables into `awk` with the `-v` option, which assigns a value before processing begins. For example, `threshold=100; awk -v t="$threshold" '$3 > t {print $0}' file.txt` makes the value available inside `awk` as the variable `t`.

7. Is AWK suitable for large file processing?

Yes. `awk` processes input one record at a time and does not need to load the entire file into memory, which makes it efficient for large datasets. For very complex processing or multi-stage pipelines, other tools (such as Python) may be easier to maintain, but `awk` remains a strong option for fast extraction and aggregation.

8. How do I write and run an AWK program stored in a file?

You can put an `awk` program in a plain text file (for example, `script.awk`) and run it with `awk -f script.awk input.txt`. This approach is easier to read and maintain

than a long one-liner, especially when your program uses multiple rules, functions, or blocks.

Conclusion

By now, you should have a basic understanding of how you can use the `awk` command to filter, format, and selectively print text from files and command output. You have also seen how `awk` goes beyond simple printing by using internal variables and `BEGIN / END` blocks for structured output, associative arrays for one-pass aggregation, and shell scripting patterns for repeatable automation. As you apply these techniques to real-world inputs like logs and delimited data, you will be able to choose between `awk` and `sed` based on whether your task is field-oriented data processing or line-oriented editing.

To learn more about `awk`, you can read the [free public-domain book by its creators](#) which goes into much more detail.

For more tutorials on how to manipulate text, check out the following articles:

- [The Basics of Using the Sed Stream Editor to Manipulate Text in Linux](#)
 - [Mastering grep with Regular Expressions for Efficient Text Search](#)
 - [An Introduction to Linux I/O Redirection](#)
 - [How To Use Bash's Job Control to Manage Foreground and Background Processes](#)
 - [An Introduction to Linux Basics](#)
-